

BEYOND BITS

Design, Implementation, and End-to-End Verification of a Compressed Communication Pipeline with Channel Noise and Hamming(7,4) Error Correction

Field	Detail
Project Title	Design of a Compression System for Effective Communication
Event	Summer of Innovation 2026 (SOI '26)
Organized by	Electronics Club, IIT Dharwad
Host Institution	Indian Institute of Technology Dharwad
Submission Date	12 July 2026
Enhancement Implemented	#3 -- Channel noise injection + RTL-based Hamming(7,4) error detection/correction
Pipeline Status	Fully verified end-to-end: input text recovered bit-exact after RLE compression, Hamming ECC, LTSpice ASK modulation over a noisy channel, demodulation, ECC decode, and RTL decompression

1. Executive Summary & Problem Rationale

Modern high-throughput communication systems are bounded by band-limited channels and physical channel noise. Transmitting raw, uncompressed digital data wastes bandwidth, and transmitting it over a real (noisy) channel without any protection risks silent data corruption. This report documents an end-to-end mixed-signal pipeline that combines hardware (RTL) data compression, RTL-based Hamming(7,4) forward error correction, and an LTSpice-simulated Amplitude Shift Keying (ASK) link with injected channel noise, closing the loop back to software for verification.

The system was built for the IIT Dharwad Summer of Innovation 2026 (SOI '26) “Beyond Bits” challenge and implements Pipeline Enhancement #3: addition of channel noise and RTL-based error-detection/error-correction coding. The complete pipeline was executed end-to-end for the input string “hello world” and the recovered output matched the original input exactly.

2. System Architecture & Pipeline Design

The pipeline is a nine-stage software/hardware/analog loop:

```
[1] Python: text -> 8-bit-per-char binary (bits.txt)
[2] Verilog RTL: run-length compression (rle_compressor.v -> compressed.txt)
[3] Verilog RTL: pad to a multiple of 4, Hamming(7,4) encode
    (hamming74_enc.v -> encoded_clean.txt)
[4] Python: encoded bits -> PWL waveform (signal.pwl)
[5] LTSpice: ASK modulation + injected white channel noise + envelope demodulation
    (Modulator_with_demod_FIXED.asc ->
demod_output.txt)
[6] Python: threshold/sample analog waveform back to bits (demod_bits.txt)
[7] Verilog RTL: Hamming(7,4) decode -- detects & corrects single-bit errors
    (hamming74_dec.v -> decoded_bits.txt)
[8] Verilog RTL: trim ECC padding, run-length decompression
    (decompressor.v ->
```

```
final_recovered_bits.txt)
[9] Python: bits -> text, compare against original input
```

This is an extension of the SOI '26 baseline five-stage pipeline (compress → modulate → transmit → demodulate → reconstruct); stages 3 and 7 (Hamming encode/decode) are the added Enhancement #3 stages.

3. Hardware Data Compression (RTL Implementation)

3.1 Compressor / Decompressor

A serial synchronous Run-Length Encoding (RLE) compressor was implemented in Verilog (rle_compressor.v). It tracks runs of identical input bits and, on a value change or a run-length saturation boundary (15, since the count field is 4 bits), emits a (run_bit, run_count) token. The complementary rle_decompressor module (decompressor.v) re-inflates these tokens back into the original bitstream and is exercised by two testbenches: a standalone round-trip proof (tb_decompressor_standalone_demo.v) and the final pipeline-integrated stage (tb_decompressor_final.v).

3.2 Compression Metrics -- Measured, Not Estimated

For the verified end-to-end run with input text “hello world”, the actual RTL simulation produced the following (all figures read directly from compressed.txt / bits.txt of the verified run, not estimated):

Stage	Size (bits)	Note
Original text (11 ASCII chars × 8 bits)	88	bits.txt
After RLE compression	225	compressed.txt -- 45 runs × 5 bits/run (1 value bit + 4 count bits)
Compression Ratio CR = Original / Compressed	88 / 225 = 0.39×	See discussion below

This result is reported honestly: for this particular input, RLE expands the data rather than compressing it. This is expected, mathematically correct behavior, not a bug — RLE only pays off when the input has long runs of repeated bits. “hello world” is 11 characters of mixed-case ASCII text with almost no repeated bit-runs, so the average run length is close to 1, and each token still costs a fixed 5 bits (1 value bit + 4-bit count) regardless of how short the run is. Fixed-overhead schemes like this one are well known to expand low-redundancy data — this is the same reason gzip/DEFLATE fall back to “stored” (uncompressed) blocks when a segment isn't compressible.

Practical implication: RLE is the right choice per the SOI '26 design brief (“e.g., Run-Length Encoding or similar”) and is trivial to realize in RTL, but it is only bandwidth-efficient on redundant payloads (e.g. repeated characters, sparse/blank-heavy data, run-length-friendly sensor logs). For a general text payload like natural language, a symbol-frequency scheme (e.g. Huffman) would compress better on average; this trade-off is discussed further as a possible extension in Section 8.

4. RTL-Based Error-Detection / Error-Correction Coding -- Hamming(7,4)

Pipeline Enhancement #3 was implemented as a Hamming(7,4) single-error-correcting code, added as two new RTL modules between compression and modulation.

4.1 Encoder (hamming74_enc.v)

Takes 4 data bits {d1,d2,d3,d4} and produces a 7-bit codeword by adding 3 parity bits, each covering a fixed, standard subset of the data bits:

position:	1	2	3	4	5	6	7
content:	p1	p2	d1	p3	d2	d3	d4

p1 = d1 XOR d2 XOR d4 (covers positions 1,3,5,7)
p2 = d1 XOR d3 XOR d4 (covers positions 2,3,6,7)
p3 = d2 XOR d3 XOR d4 (covers positions 4,5,6,7)

The compressed bitstream (225 bits) is first zero-padded to a multiple of 4 (228 bits, +3 pad bits, recorded in ecc_pad.txt for later trimming) and encoded 4 bits at a time, producing 57 codewords $\times$ 7 bits = 399 bits total (encoded_clean.txt). This is the data that is actually converted to the PWL waveform and sent through the LTSpice channel.

4.2 Decoder (hamming74_dec.v)

Recomputes the three parity checks on the received 7-bit codeword. A non-zero 3-bit syndrome directly encodes the position (1–7) of a single flipped bit, which is then inverted before the 4 data bits are extracted. This corrects any single-bit error introduced anywhere along the channel — compression token corruption, modulation threshold errors, or channel noise — without retransmission.

4.3 Verification: Deliberate Fault Injection

Because a clean channel run may not naturally produce an error, correctness of the ECC decoder was additionally verified with a dedicated fault-injection testbench (tb_ecc_correction_demo.v) that encodes the same 57-codeword stream, deliberately flips one bit, and confirms the decoder both detects and corrects it. Actual output from this test:

```
ECC Verification Report -- Hamming(7,4)
=====
Total codewords: 57
Deliberately flipped bit index 4 of codeword #28 (simulated channel noise)

>> Injected bit flip into codeword #28: clean=0100101 corrupted=0110101
Codeword #28: ERROR DETECTED at bit position 3 -- CORRECTED (recovered data=0101)

Encode -> corrupt -> decode cycle complete.
```

The decoder correctly localized the fault to bit position 3 and recovered the exact original 4-bit data nibble (0101), confirming the Hamming(7,4) implementation's single-bit correction capability independent of whatever the real channel happened to do on any one run.

4.4 Cost of the Enhancement

Adding Hamming(7,4) protection costs a fixed 75% bit overhead (7 transmitted bits per 4 data bits): the 228 padded compressed bits become 399 transmitted bits. This is the standard, expected cost of single-error-correction coding and is quantified here rather than left as an unstated assumption.

5. Physical Layer Analog Simulation (LTSpice)

5.1 Modulator

The 399-bit encoded stream is converted to a piecewise-linear (PWL) voltage waveform (signal.pwl, 5V logic-high / 0V logic-low, 1 μ s per bit $\rightarrow$ 1 Mbps) and drives an idealized voltage-controlled switch (S1) that gates a 10 MHz sine carrier onto the channel -- standard ASK/OOK modulation:

```
.model S1 sw(Vt=2.5 Vh=0.5 Ron=0.1 Roff=1MEG)
V1: SINE(0 1 10MEG)            -> 10 carrier cycles per 1 us bit
```

5.2 Channel Noise (Enhancement Requirement)

A behavioral voltage source injects pseudo-random white noise in series with the modulated signal before the receiver, per the SOI '26 requirement to add channel noise using LTSpice or MATLAB/Python:

```
V = white(1e6 * time) * {NOISE_AMP}
.param NOISE_AMP = 0.3
```

5.3 Demodulator

A diode (1N4148) + RC envelope detector (1 k Ω / 470 pF) recovers the baseband envelope at node DEMOD_OUT, which is exported as demod_output.txt and thresholded in Python back into bits.

6. Channel Performance: BER & SNR Analysis

This section is included per the SOI '26 requirement that BER and SNR analysis be reported when Enhancement #3 is implemented. All figures below are computed directly from the verified run's demod_output.txt and demod_bits.txt (399 transmitted bits, NOISE_AMP = 0.3).

Metric	Value	How it was obtained
Logic-'1' sample cluster	mean 0.366 V, σ = 0.056 V	Max voltage in a guarded sampling window per bit, over all bits Python identified as '1'
Logic-'0' sample cluster	mean 0.007 V, σ = 0.009 V	Same, over bits identified as '0'
Adaptive decision threshold	0.241 V	Midpoint between observed low (0.000 V) and high (0.481 V) extremes
Estimated channel SNR	$\approx$ 14.9 dB	$10 \cdot \log_{10}[(\text{signal amplitude} / 2\sigma_{\text{avg}})^2]$, signal amplitude = 0.360 V, σ_{avg} = 0.032 V
Bits transmitted	399	tx_bit_count.txt
Raw bit errors (pre-ECC, this run)	0 / 399 (BER = 0)	demod_bits.txt vs encoded_clean.txt, bit-exact match
Codeword errors detected/corrected (this run)	0 / 57 codewords	ecc_decode_report.txt

At NOISE_AMP = 0.3, the '0' and '1' voltage clusters are well separated relative to their spread (0.007 V vs 0.366 V, against a combined noise σ of about 0.03 V), giving an estimated post-demodulation SNR of $\sim$ 14.9 dB and zero bit errors over the full 399-bit transmission in this run -- the system has comfortable margin at this noise level. Because a clean run alone cannot demonstrate correction capability, Section 4.3's deliberate fault-injection test is the complementary evidence that the Hamming(7,4) decoder does correct a single-bit error when the channel does produce one: it detected and corrected a forced flip at codeword #28 with zero residual error.

Recommended follow-up (optional, not required for this submission): sweeping NOISE_AMP upward (e.g. 0.6, 1.0) until the natural channel BER becomes non-zero would let this section show a real BER-vs-SNR trend and the point at which single-bit correction is no longer sufficient (i.e. when a codeword suffers 2+ errors). The pipeline as built supports this directly by re-running LTSpice at a higher NOISE_AMP and re-executing Steps 6–9 of the notebook.

7. Software Data Recovery & End-to-End Verification

The corrupted/attenuated waveform at DEMOD_OUT is exported to Python, sampled with an adaptive threshold, Hamming-decoded in RTL, trimmed of ECC padding, RTL-decompressed, and converted back to text. Actual measured results for the full verified run:

Stage	Output	Result
1. Source text	"hello world" -> 88 bits	Pass
2. RTL RLE compression	225 bits (45 runs)	Pass -- verified via testbench
3. Pad + Hamming(7,4) encode	228 -> 399 bits (57 codewords)	Pass
4. ASK modulation + white channel noise (NOISE_AMP=0.3)	10 MHz carrier, 1 Mbps burst	Pass
5. Envelope demodulation + adaptive thresholding	399 bits recovered, threshold 0.241 V	Pass -- bit-exact vs encoded_clean.txt
6. RTL Hamming(7,4) decode	57/57 codewords, 0 corrections needed this run	Pass (see §4.3 for correction-capability proof)
7. Trim padding + RTL decompression	225 -> 88 bits	Pass
8. Bits -> text	"hello world"	Pass -- exact match to source

Final automated verification printout from the notebook (Step 9):

```
===== END-TO-END VERIFICATION =====
Expected Input:  'hello world'
Recovered Text:  'hello world'
Pipeline Integrity Verified: True
=====
```

8. Conclusion

The integrated pipeline validates the full loop from software text generation through hardware compression, RTL-based Hamming(7,4) forward error correction, LTSpice analog modulation and channel-noise injection, envelope demodulation, and RTL decompression, recovering the source text bit-exactly. Two results stand out for discussion: first, RLE compression ratio is strongly input-dependent -- it expanded this particular low-redundancy 11-character input ($CR = 0.39\times$) while being well suited to repetitive payloads, which is a meaningful and honestly reported engineering trade-off rather than a shortcoming to hide. Second, the Hamming(7,4) enhancement was verified both passively (0 channel-induced errors at an estimated 14.9 dB SNR) and actively (a deliberately injected single-bit fault at codeword #28 was correctly detected and corrected), giving confidence in the decoder's correction logic beyond what a single noise-free run alone could show.

Possible future extensions: a symbol-frequency compressor (Huffman) for better average-case compression on natural-language payloads, and a BER-vs-SNR sweep across multiple NOISE_AMP values to characterize the single-bit-correction limit of the Hamming(7,4) scheme.